Telco Customer Churn Prediction

Midterm Project Report

MSITM 6341: Python

Carter School of Business, Dallas Baptist University

Group B

Rommel Mahabir

Sarvagya Dubey

Rajendra Konduru

Rejin Sudarsanan Reji

Instructor: Dean-Yuan (Dennis) Wang

November 1, 2026

Repository: github.com/rommelmahabir/MSITM6341-GroupB-Midterm

Contents

1. Introduction to the Dataset..... 3
2. Data Cleaning & Preprocessing..... 4
3. EDA & Key Insights..... 6
4. Model Building & Evaluation..... 10
5. Conclusion..... 13
6. References..... 14
Appendix A: Data Dictionary..... 15
Appendix B: Use of Generative AI..... 17

1. Introduction to the Dataset

For this project we used the Telco Customer Churn dataset. It is sample data made by IBM about a made-up telecom company. Each row is one customer, with details about their account and the services they pay for.

We got the dataset from Kaggle (kaggle.com/datasets/blastchar/telco-customer-churn). We downloaded it as one CSV file on October 25, 2026 and saved it in our repo as `data/WA_Fn-UseC_-Telco-Customer-Churn.csv`. We did not change the file itself. All cleaning is done in our Python code. The license on Kaggle is listed as “Data files © Original Authors”.

The data has 7,043 customers and 21 columns. Some columns are about the customer, like gender, whether they are a senior citizen, and whether they have a partner or dependents. Others are about their account, like how many months they have been a customer, their contract type, how they pay, and their monthly and total charges. The rest are about phone and internet services and add-ons like online security, tech support and streaming. There is also a customer ID, which we dropped because it is just a label.

We want to predict Churn, which means whether a customer left the company in the last month. About 26.5% of customers left, so around 1 in 4. This matters because it usually costs a telecom company more to get a new customer than to keep one. If the company knows who is likely to leave, it can try to keep them with better deals or support.

2. Data Cleaning & Preprocessing

All cleaning and preprocessing happens in `data_prep.py`. The raw CSV is never edited, so every change can be traced and re-run from the original file. Two functions hand the data to the rest of the project:

- `load_clean_data()` returns the cleaned table in readable form (for example, "Month-to-month"), used for EDA in Section 3.
- `get_model_data()` returns the encoded, scaled training and test sets used by both models in Section 4.

Each decision below is also recorded in `DECISION_LOG.md`, and every column is described in the data dictionary.

2.1 Data Cleaning

The raw file has 7,043 rows and 21 columns.

Missing values. `TotalCharges` loaded as text instead of numbers, because 11 rows contained a blank space. All 11 belonged to customers with a tenure of 0 months: new customers who had not yet been billed. Their true total is therefore 0, so we set the blanks to 0 rather than dropping the rows (which would lose real customers) or filling them with the average (which would invent charges they never paid). The column was then converted to a number. No other column had missing values.

Duplicates. We checked for duplicates while `customerID` was still present: 7,043 rows and 7,043 unique IDs, with no duplicate rows, so nothing was removed. Once the ID is dropped, 22 rows share identical details, but they are different customers who happen to have the same profile, so we kept them.

Invalid entries. We checked every text column's values. Each has between 2 and 4 expected categories (for example, Yes / No / No internet service), with no typos or unexpected values, and `Churn` contains only "Yes" and "No".

Irrelevant columns. `customerID` was dropped. It labels a customer but does not describe them, so it cannot help predict churn.

The cleaned table has **7,043 rows and 20 columns**.

2.2 Preprocessing

The models need numbers on comparable scales, so we applied four steps:

Step	What we did	Why
Target encoding	`Churn`: Yes became 1, No became 0	Models predict numbers. 26.5% of customers churned, so the classes are imbalanced
One-hot encoding	The 15 text columns became 26 columns of 0s and 1s, using `pd.get_dummies(drop_first=True)`	Gives each category its own column without implying an order. Dropping one category per column avoids duplicate information (the dummy variable trap)

Step	What we did	Why
Train/test split	80% training (5,634 customers), 20% test (1,409), stratified, `random_state=42`	Stratifying keeps the 26.5% churn rate in both sets. The fixed seed means both models use the identical split
Scaling	`tenure`, `MonthlyCharges` and `TotalCharges` standardized with `StandardScaler`, fit on the training set only	These range from 0 to over 8,000, while the other features are 0 or 1. Fitting on training data only prevents information about the test set leaking into training

The result is **30 feature columns**: the 26 one-hot columns plus `SeniorCitizen` (already 0/1), `tenure`, `MonthlyCharges` and `TotalCharges`.

2.3 Testing

`test_data_prep.py` checks that the cleaning worked: 7,043 rows and 20 columns, no missing values, `TotalCharges` stored as a number, `customerID` removed, and `Churn` containing only Yes and No. All five checks pass.

3. EDA & Key Insights

3.1 Overview of the Data

The dataset contains 7,043 customer records and 20 columns covering customer demographics, services, contract information, monthly charges, total charges, and churn status. The initial data inspection using df.info() showed that all columns contain 7,043 non-null values, indicating no missing values after cleaning. The numerical variables include SeniorCitizen, tenure, MonthlyCharges, and TotalCharges. Customer tenure ranges from 0 to 72 months, with an average of approximately 32.4 months. Monthly charges range from $18.25 to $118.75, with an average of $64.76. Total charges average approximately $2,279.73.

The target variable, Churn, is imbalanced: 5,174 customers (73.5%) stayed, while 1,869 customers (26.5%) left. This imbalance is important when evaluating classification models because accuracy alone may not adequately measure how well a model identifies customers who churn.

3.2 Key Insights

1. Customer Churn Distribution

The churn distribution shows that most customers stayed with the company, while approximately 26.5% discontinued their service. This class imbalance is important for model development because the classification models should identify customers who leave rather than simply favoring the larger group of customers who stay.

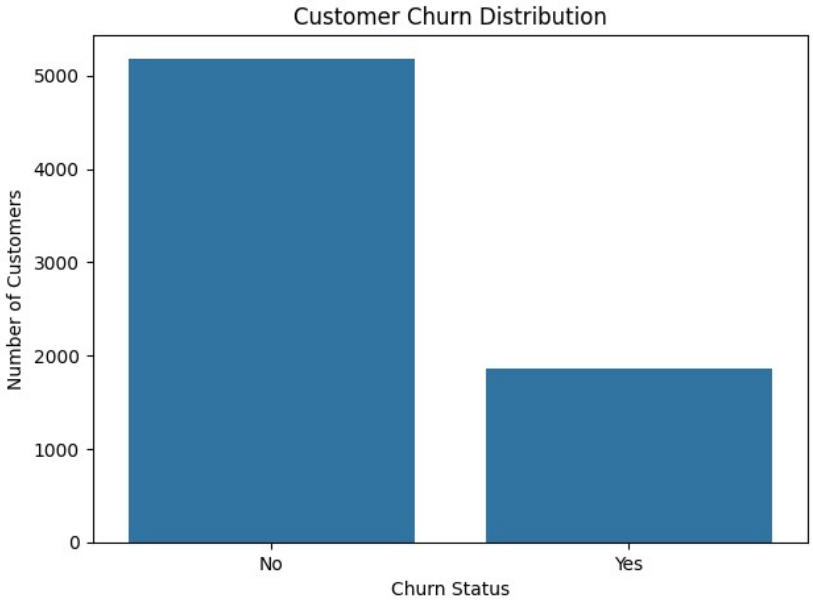


Figure 3.1: Customer churn distribution

2. Tenure by Churn

The tenure histogram shows that churn is concentrated among customers with shorter tenure. Customers who have stayed with the company for longer periods are more likely to remain customers.

This suggests that the early stages of the customer relationship may be an important period for retention efforts.

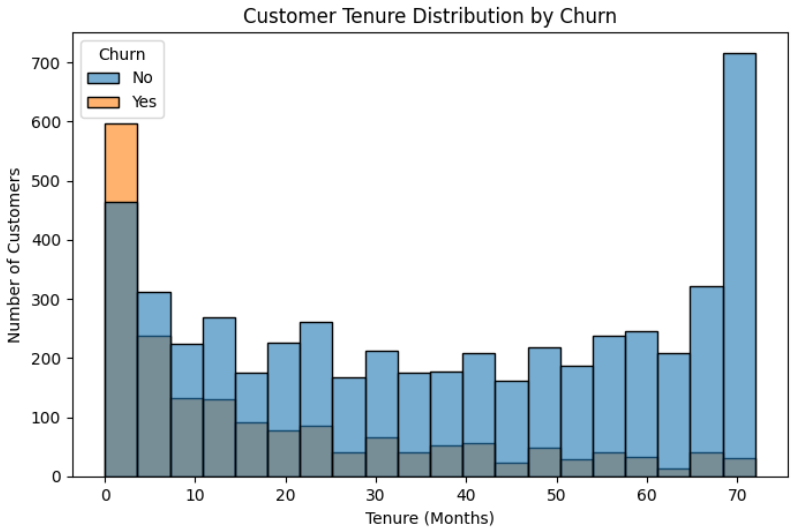


Figure 3.2: Customer tenure distribution by churn

3. Monthly Charges by Churn

The box plot indicates that customers who churn generally pay higher monthly charges than customers who stay. The median monthly charge is approximately $80 for customers who left, compared with approximately $64 for customers who stayed. This suggests that monthly pricing may be associated with customer retention.

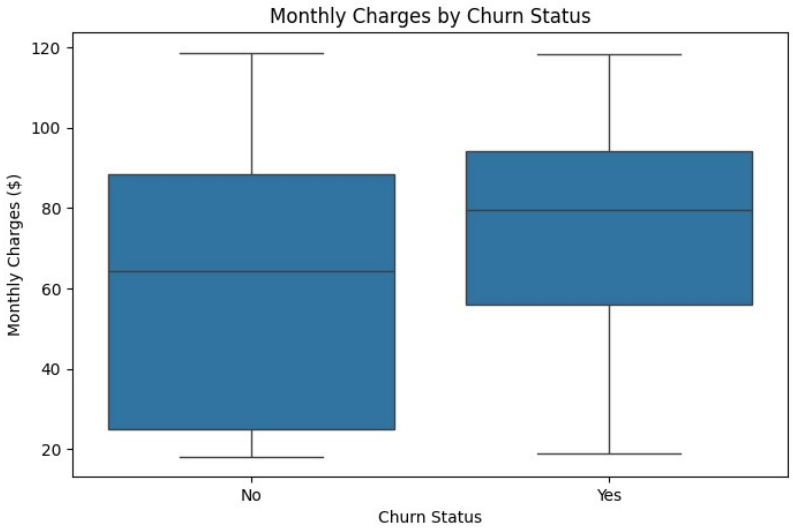


Figure 3.3: Monthly charges by churn status

4. Churn Rate by Contract

Contract type shows a substantial difference in churn rates. Month-to-month customers have the highest churn rate at approximately 42.7%, compared with 11.3% for one-year contracts and 2.8% for

two-year contracts. Customers with longer-term contracts appear more likely to remain with the company.

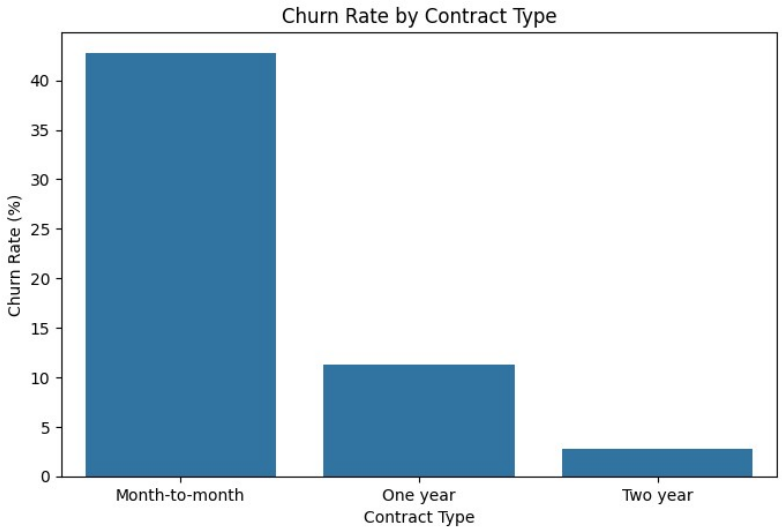


Figure 3.4: Churn rate by contract type

5. Correlation Heatmap

The correlation heatmap shows a strong positive relationship between tenure and TotalCharges (approximately 0.83). This is reasonable because customers who remain longer generally accumulate higher total charges. MonthlyCharges and TotalCharges also show a positive correlation (approximately 0.65). These relationships help explain how customer duration and spending patterns are connected.

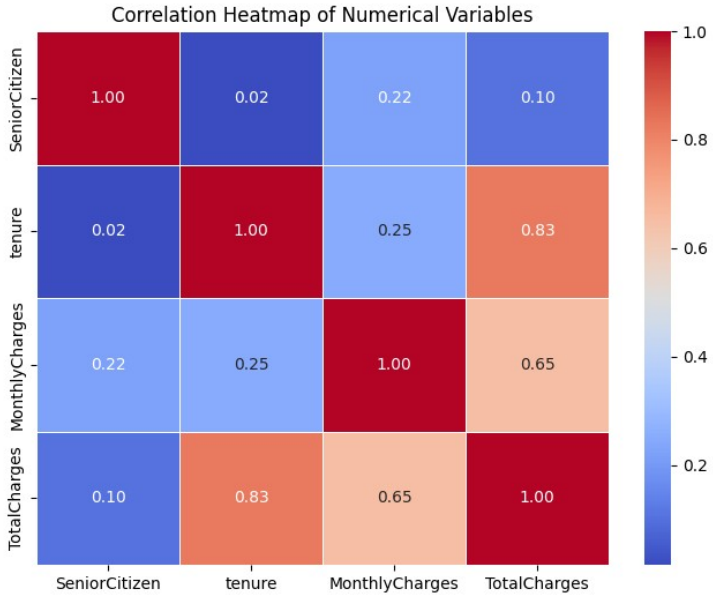


Figure 3.5: Correlation heatmap of numerical variables

3.3 Summary

Overall, the EDA highlights customer tenure and contract type as important patterns associated with churn. The higher churn rate among month-to-month customers and the concentration of churn among newer customers suggest useful areas for retention strategies. These findings also align with the Section 4 model results, where tenure and contract type were among the strongest predictors in both Logistic Regression and Random Forest. The EDA therefore provides a useful foundation for understanding and interpreting the classification models.

4. Model Building & Evaluation

4.1 Approach

Both models were trained and tested on exactly the same data, using the shared `get_model_data()` function from Section 2: 30 encoded features, a stratified 80/20 split (5,634 training and 1,409 test customers), and a fixed random seed. This makes their scores directly comparable.

Because only 26.5% of customers churn, accuracy alone is misleading: a model that predicts "No churn" for everyone would score 73.5% while catching no churners at all. We therefore report four metrics for the churn class:

- **Accuracy:** the share of all predictions that were correct
- **Precision:** of the customers flagged as churners, the share who actually left
- **Recall:** of the customers who actually left, the share the model caught
- **F1:** a single score balancing precision and recall

For a churn problem, **recall matters most**. Missing a customer who is about to leave costs more than offering a retention deal to someone who would have stayed.

4.2 Model 1: Logistic Regression

We started with Logistic Regression as a simple, interpretable baseline (`max_iter=1000`). During the review of our preprocessing, Rajendra suggested trying `class_weight="balanced"`, which gives the rarer churners more weight during training. We trained both versions:

Version	Accuracy	Precision	Recall	F1
Default	0.806	0.659	0.559	0.605
Balanced	0.738	0.504	**0.783**	**0.614**

The default model caught only 209 of the 374 churners in the test set and missed 165. Balanced weights raised recall from 56% to 78%. Accuracy fell because the model now flags more customers, including some who stay, but F1 improved slightly, and the model finds far more of the customers who matter.

Logistic Regression also shows the direction of each feature's effect. The strongest signals toward churn were **Fiber optic internet** and **electronic check payments**; the strongest away from churn were **two-year contracts**, **one-year contracts** and **longer tenure**. Some coefficients should be read with care: features such as MonthlyCharges and TotalCharges overlap heavily with the service and tenure columns (multicollinearity), so individual coefficients show patterns rather than exact effects.

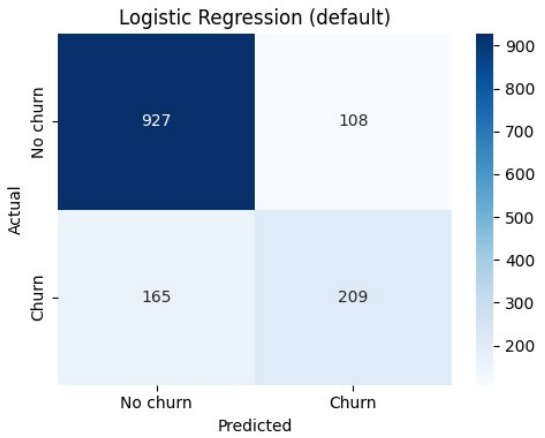


Figure 4.1: Logistic Regression (default): confusion matrix

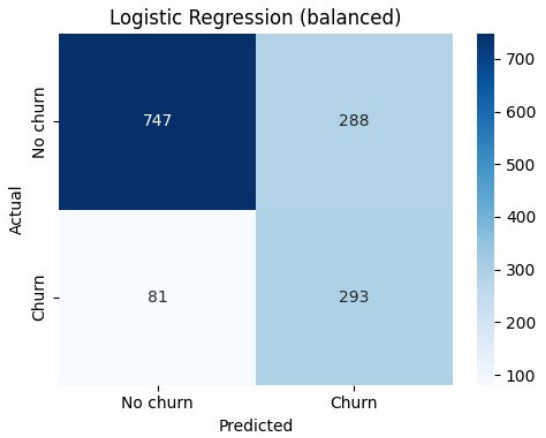


Figure 4.2: Logistic Regression (balanced): confusion matrix

4.3 Model 2: Random Forest

Random Forest combines many decision trees and can capture interactions between features that Logistic Regression can't. We trained three versions:

Version	Accuracy	Precision	Recall	F1
Default	0.789	0.629	0.503	0.559
Balanced	0.791	0.637	0.492	0.555
Tuned	0.760	0.536	**0.725**	**0.616**

Unlike with Logistic Regression, balanced weights alone barely changed the default Random Forest. Its trees grow without a depth limit until every leaf is pure, so the weights have almost nothing to act on.

We then tuned the model with `GridSearchCV`, testing 12 combinations of tree count, maximum depth and minimum leaf size. Each combination was scored on F1 using 5-fold cross-validation on the **training set only**, so the test set was never used for tuning. The best settings were `max_depth=10`, `min_samples_leaf=1` and `n_estimators=200` (cross-validated F1 of 0.636). Limiting the depth let the balanced weighting work: recall rose from 0.50 to 0.73. The tuned model caught 271 of the 374 churners. Random Forest results can differ slightly in the third decimal place between scikit-learn versions, because the trees are built from random samples; the figures above come from our development environment, and the conclusions are the same in either case.

The features the forest relied on most were **tenure**, **TotalCharges**, **MonthlyCharges**, **two-year contracts** and **Fiber optic internet**.

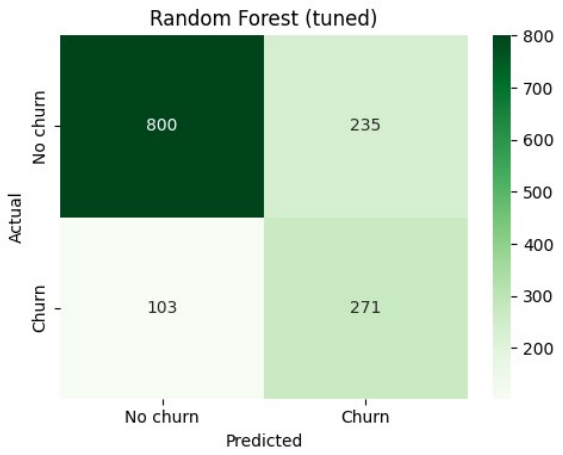


Figure 4.3: Random Forest (tuned): confusion matrix

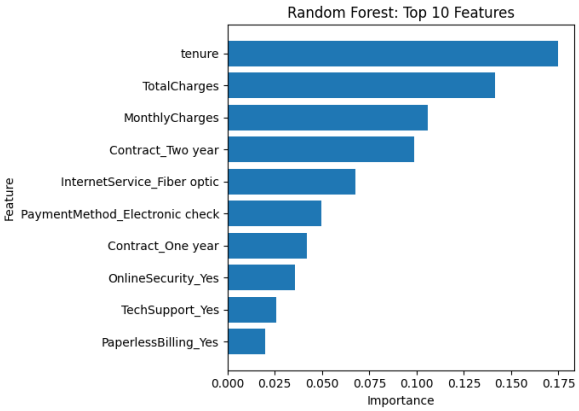


Figure 4.4: Random Forest (tuned): top 10 feature importances

4.4 Comparison and Recommended Model

Model	Accuracy	Precision	Recall	F1
Logistic Regression (balanced)	0.738	0.504	**0.783**	0.614
Random Forest (tuned)	0.760	0.536	0.725	**0.616**

The two models are effectively tied on F1. We recommend **Logistic Regression (balanced)** because it catches more of the customers who actually leave (78% vs 72%), it is simpler and faster, and it shows whether each factor increases or decreases churn risk. The more complex model did not earn its extra complexity on this dataset.

Both models agree on the main drivers of churn: **short tenure**, **month-to-month contracts** and **Fiber optic internet**. The main trade-off is precision: about half the customers the recommended model flags would not actually have left, so any retention offer should be low-cost enough to justify those false alarms.

5. Conclusion

The goal of this project was to figure out which telecom customers are likely to leave. We used the Telco Customer Churn dataset and built two models, Logistic Regression and Random Forest, both tested on the same data split.

Our pick is Logistic Regression with balanced class weights. The two models ended up almost tied on F1 (0.614 vs 0.616), but Logistic Regression caught more of the customers who actually left, 78% compared to 72%. It's also simpler, and it's easier to see how each factor affects churn. Balancing the class weights was a big part of this, since it pushed recall up from 56% to 78%.

Both models pointed to the same three reasons customers leave:

- **Tenure:** new customers are the most likely to leave. Extra support or a small welcome offer in the first few months could help.
- **Contract type:** month-to-month customers leave a lot more than those on two-year contracts. Offering a discount to switch to a longer contract could keep more of them.
- **Fiber optic internet:** these customers churn more than others. It would be worth looking into whether the price or the service is the problem.

This is sample data from a made-up company, so real results could look different. The model also flags a lot of customers who end up staying, since its precision is only about 0.50. We're okay with that, because losing a customer costs more than sending someone an offer they didn't need.

In the future we can try other models like gradient boosting, do more tuning, and test on real or more recent customer data.

6. References

- **Dataset:** Telco Customer Churn (IBM sample data), Kaggle: https://www.kaggle.com/datasets/blastchar/telco-customer-churn. Also published by IBM on GitHub under the Apache 2.0 license: https://github.com/IBM/telco-customer-churn-on-icp4d
- **pandas:** https://pandas.pydata.org
- **NumPy:** https://numpy.org
- **scikit-learn:** https://scikit-learn.org
- **matplotlib:** https://matplotlib.org
- **seaborn:** https://seaborn.pydata.org

Appendix A: Data Dictionary

The raw file `data/WA_Fn-UseC_-Telco-Customer-Churn.csv` has 7,043 rows (one per customer) and 21 columns. Row numbers in brackets refer to `DECISION_LOG.md`.

Column	What it means	Raw type	How we cleaned or encoded it
customerID	Unique ID for each customer	Text	Dropped: it labels a customer but doesn't describe them (Decision Log row 3)
gender	Customer's gender: Male or Female	Text	One-hot encoded into 1 column (row 5)
SeniorCitizen	Whether the customer is a senior citizen: 1 = yes, 0 = no	Number (0/1)	Unchanged: already 0/1
Partner	Whether the customer has a partner or spouse	Yes/No	One-hot encoded into 1 column (row 5)
Dependents	Whether the customer has dependents, such as children	Yes/No	One-hot encoded into 1 column (row 5)
tenure	Number of months the customer has been with the company (0–72)	Number	Scaled with StandardScaler, fit on the training set only (row 7)
Contract	Contract length: Month-to-month, One year or Two year	Text	One-hot encoded into 2 columns (row 5)
PaperlessBilling	Whether the customer gets bills online instead of on paper	Yes/No	One-hot encoded into 1 column (row 5)
PaymentMethod	How the customer pays: Electronic check, Mailed check, Bank transfer (automatic) or Credit card (automatic)	Text	One-hot encoded into 3 columns (row 5)
MonthlyCharges	Amount billed each month, in dollars (about $18–$119)	Number	Scaled with StandardScaler, fit on the training set only (row 7)
TotalCharges	Total amount billed to date	Text	11 blanks (all tenure 0) set to 0, converted to a number, then scaled (rows 1, 7)
PhoneService	Whether the customer has phone service	Yes/No	One-hot encoded into 1 column (row 5)
MultipleLines	Whether the customer has more than one phone line: Yes, No, or No phone service (for customers without phone service)	Text	One-hot encoded into 2 columns (row 5)
InternetService	Type of internet service: DSL, Fiber optic, or No	Text	One-hot encoded into 2 columns (row 5)
OnlineSecurity	Whether the customer has the online security add-on: Yes, No, or No internet service	Text	One-hot encoded into 2 columns (row 5)

Column	What it means	Raw type	How we cleaned or encoded it
OnlineBackup	Whether the customer has the online backup add-on: Yes, No, or No internet service	Text	One-hot encoded into 2 columns (row 5)
DeviceProtection	Whether the customer has the device protection add-on: Yes, No, or No internet service	Text	One-hot encoded into 2 columns (row 5)
TechSupport	Whether the customer has the tech support add-on: Yes, No, or No internet service	Text	One-hot encoded into 2 columns (row 5)
StreamingTV	Whether the customer streams TV: Yes, No, or No internet service	Text	One-hot encoded into 2 columns (row 5)
StreamingMovies	Whether the customer streams movies: Yes, No, or No internet service	Text	One-hot encoded into 2 columns (row 5)
Churn	Whether the customer left in the last month	Yes/No	Target: Yes = 1, No = 0 (row 4)

Notes

- One-hot encoding uses `drop_first=True`, so a column with 2 values becomes 1 column and a column with 3 values becomes 2. The 15 text columns become 26 columns; with SeniorCitizen, tenure, MonthlyCharges and TotalCharges, the models use 30 feature columns.
- "No phone service" and "No internet service" mean the customer doesn't have the base service, so the add-on doesn't apply. That is why those columns have 3 values instead of 2.
- No duplicate customers were found, so no rows were removed (row 2).

Appendix B: Use of Generative AI

The course allows the use of generative AI (GenAI) on assignments and group projects, provided its use is documented in detail. Our team used AI tools, including Claude, ChatGPT and Gemini, to help plan the project, write and debug code, and draft documentation and report sections.

Every member reviewed, ran and edited the AI-assisted work they contributed, and can explain it. Each cleaning, preprocessing and modeling decision, with its reason, is recorded in `DECISION_LOG.md`.

Each member's detailed AI use log is in the project repository:

- `ai-use/rommel.md`
- `ai-use/sarvagya.md`
- `ai-use/rajendra.md`
- `ai-use/rejin.md`